

Comprehensive Computer Networking Notes: Routing Algorithms

Intradomain vs. Interdomain Architecture, Distance Vector Disadvantages, Count-to-Infinity Problem, Split Horizon, Poison Reverse, and Solved Conceptual Exam Problems

Computer Networking Academic Series

September 4, 2026

Abstract

This document provides an in-depth analytical exploration of network routing protocols, focusing on Intradomain (IGP) vs. Interdomain (EGP) routing and Autonomous System (AS) governance. It presents a detailed comparison between Distance Vector (DV) and Link State (LS) paradigms, rigorously analyzing why DV is a decentralized protocol. Furthermore, it details the architectural limitations of Distance Vector protocols, the mathematical mechanics of the *Count-to-Infinity* routing loop problem, and solutions including *Split Horizon*, *Poison Reverse*, and timers. Finally, it integrates step-by-step solutions to slide-based conceptual exam questions regarding neighbor state tracking, Hello packet exchange omission, and initial routing vector updates across complex multi-router topologies.

Contents

1 Architectural Domains: Intradomain vs. Interdomain Routing	3
1.1 Autonomous Systems (AS)	3
1.2 Intradomain vs. Interdomain Routing Protocols	3
2 Algorithmic Paradigms: Distance Vector vs. Link State	4
2.1 Decentralized Architecture of Distance Vector (DV)	4
2.2 Distance Vector (DV) vs. Link State (LS) Comparison	4
3 Step-by-Step Distance Vector Simulation: Cold Start to Convergence	4
3.1 Network Topology Setup	4
3.2 Stage 0: Cold Start ($t = 0s$)	4
3.3 Stage 1: First Routing Exchange ($t = 30s$)	5
3.4 Stage 2: Full Network Convergence ($t = 60s$)	5
4 Architectural Disadvantages & The Count-to-Infinity Problem	6
4.1 Major Architectural Disadvantages of Distance Vector	6
4.2 Mathematical Analysis of the Count-to-Infinity Problem	6
4.3 Distance Vector Iteration Table (Count-to-Infinity Trace)	6
5 Solutions to Routing Loops	6
5.1 1. Split Horizon Rule	6
5.2 2. Poison Reverse (Split Horizon with Poison Reverse)	7
5.3 3. Timers and Route Poisoning	7

6	Solutions to Conceptual Exam Questions (Lecture Slides)	7
6.1	Question 1: Neighbor Tracking and Failure Detection	7
6.2	Question 2: Absence of Hello Packets	8
6.3	Question 3: Topology-Based First Iteration Vector Determination	8

1 Architectural Domains: Intradomain vs. Interdomain Routing

In modern network architecture, the global Internet is structured as a hierarchical "network of networks." To maintain global scalability, routing administration is partitioned into administrative boundaries known as Autonomous Systems.

1.1 Autonomous Systems (AS)

An **Autonomous System (AS)** is a unified collection of IP networks and routers under the administrative control of a single organization (such as an ISP, enterprise, or university) that presents a common, clearly defined routing policy to the global Internet.

- **Global Identification & Allocation:** Each Autonomous System is identified globally by a unique 16-bit or 32-bit **Autonomous System Number (ASN)** assigned by IANA through Regional Internet Registries (RIRs like ARIN, RIPE NCC, APNIC).
- **Public IP Block Allocation:** To advertise routes globally, an AS obtains public IPv4/IPv6 address prefixes directly from its assigned RIR.
- **Internal Autonomy:** Within its boundary, an AS possesses complete autonomy to choose internal routing protocols, metrics, bandwidth allocations, and traffic engineering constraints.

1.2 Intradomain vs. Interdomain Routing Protocols

Routing protocols are divided into two distinct domain tiers based on whether they operate internally within an AS or externally between separate Autonomous Systems:

Table 1: Intradomain (IGP) vs. Interdomain (EGP) Routing Protocols

Dimension	Intradomain Routing (IGP)	Interdomain Routing (EGP)
Scope	Internal to a single Autonomous System (AS).	Between distinct Autonomous Systems.
Primary Goal	Maximum performance, minimal latency, optimal interior path calculation.	Policy enforcement, reachability, path-vector trust management.
Administrative Control	Managed by a single administrative entity.	Coordinates across multiple un-trusted administrative domains.
Common Protocols	Routing Information Protocol (RIP), Open Shortest Path First (OSPF), Enhanced Interior Gateway Routing Protocol (EIGRP).	Border Gateway Protocol (BGP-4).
Metrics Used	Hop count (RIP), Link bandwidth/cost (OSPF), Delay/Load (EIGRP).	AS-PATH length, administrative preferences, BGP attributes.

2 Algorithmic Paradigms: Distance Vector vs. Link State

2.1 Decentralized Architecture of Distance Vector (DV)

Distance Vector routing is fundamentally a **decentralized protocol**.

Why Distance Vector is Decentralized:

- **Localized Path Computation:** Each router independently calculates its optimal routes using only localized updates received from its immediate physical neighbors.
- **Absence of Global Information Exchange:** No single router possesses a complete topological map of the entire network. Routing decisions are made strictly via "routing by rumor."

2.2 Distance Vector (DV) vs. Link State (LS) Comparison

Table 2: Architectural Comparison: Distance Vector vs. Link State Protocols

Feature	Distance Vector (DV)	Link State (LS)
Topology Knowledge	Localized knowledge of neighbors only ("Routing by rumor").	Complete global topology map maintained by every router.
Core Algorithm	Bellman-Ford Algorithm.	Dijkstra's Shortest Path First (SPF) Algorithm.
Update Type	Full table updates sent periodically to direct neighbors.	Event-triggered Link-State Advertisements (LSAs) flooded to all routers.
Keepalive Method	Periodic full updates act as implicit keepalives (No Hello packets).	Explicit, lightweight Hello Packets exchanged frequently.
Convergence Speed	Slow; susceptible to routing loops and count-to-infinity.	Fast; immune to count-to-infinity.

3 Step-by-Step Distance Vector Simulation: Cold Start to Convergence

To understand how distance vector tables evolve, consider a linear 3-router topology consisting of **Router 1 (R1)**, **Router 2 (R2)**, and **Router 3 (R3)**.

3.1 Network Topology Setup

3.2 Stage 0: Cold Start ($t = 0s$)

At boot up, each router only knows its directly attached networks via active physical interfaces (Hop count = 0).

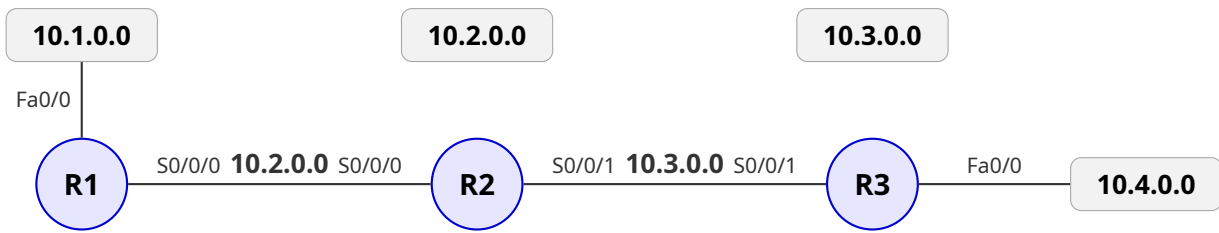


Figure 1: 3-Router Testbed Topology with FastEthernet and Serial Interfaces.

Initial State (Cold Start): Knowing Directly Connected Networks

R1 Routing Table

Network	Interface	Hop
10.1.0.0	Fa0/0	0
10.2.0.0	S0/0/0	0

R2 Routing Table

Network	Interface	Hop
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/1	0

R3 Routing Table

Network	Interface	Hop
10.3.0.0	S0/0/1	0
10.4.0.0	Fa0/0	0

3.3 Stage 1: First Routing Exchange ($t = 30s$)

After the 30-second timer, direct neighbors exchange initial vectors. R1 learns 10.3.0.0 (1 hop); R3 learns 10.2.0.0 (1 hop); R2 learns 10.1.0.0 (1 hop) and 10.4.0.0 (1 hop).

First Exchange of Routing Information ($t = 30s$)

R1 Routing Table

Network	Interface	Hop
10.1.0.0	Fa0/0	0
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/0	1

R2 Routing Table

Network	Interface	Hop
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/1	0
10.1.0.0	S0/0/0	1
10.4.0.0	S0/0/1	1

R3 Routing Table

Network	Interface	Hop
10.3.0.0	S0/0/1	0
10.4.0.0	Fa0/0	0
10.2.0.0	S0/0/1	1

3.4 Stage 2: Full Network Convergence ($t = 60s$)

At $t = 60s$, R2 forwards its updated table. R1 learns 10.4.0.0 at cost $1 + 1 = 2$ hops; R3 learns 10.1.0.0 at cost $1 + 1 = 2$ hops.

Second Exchange - Complete Network Convergence ($t = 60s$)

R1 Routing Table

Network	Interface	Hop
10.1.0.0	Fa0/0	0
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/0	1
10.4.0.0	S0/0/0	2

R2 Routing Table

Network	Interface	Hop
10.2.0.0	S0/0/0	0
10.3.0.0	S0/0/1	0
10.1.0.0	S0/0/0	1
10.4.0.0	S0/0/1	1

R3 Routing Table

Network	Interface	Hop
10.3.0.0	S0/0/1	0
10.4.0.0	Fa0/0	0
10.2.0.0	S0/0/1	1
10.1.0.0	S0/0/1	2

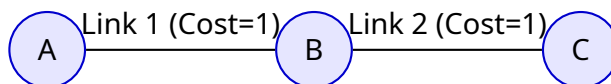
4 Architectural Disadvantages & The Count-to-Infinity Problem

4.1 Major Architectural Disadvantages of Distance Vector

1. **Local Knowledge Limitation:** Routers possess no global network map, restricting traffic engineering capabilities.
2. **Vulnerability to Misinformation (Routing by Rumor):** Routers accept updates blindly from direct neighbors without independent verification, making them vulnerable to man-in-the-middle attacks or misconfiguration propagation.
3. **Slow Convergence Rate:** Step-by-step neighbor propagation results in high convergence latency across large enterprise networks.
4. **Routing Loops & Count-to-Infinity:** Slow propagation can cause adjacent routers to form feedback loops when a link fails.

4.2 Mathematical Analysis of the Count-to-Infinity Problem

Consider three linearly linked routers: Router A $\longleftrightarrow$ Router B $\longleftrightarrow$ Router C.



- **Normal State:** Router A is connected to Subnet X. B reaches X via A at cost 1. C reaches X via B at cost 2.
- **Link Failure** ($t = 12 : 00$): Link between A and B fails. Router B updates its distance to X as ∞ (unreachable).
- **Race Condition** ($t = 12 : 15$): Before B can transmit its update to C, C's regular 30-second timer fires ($t = 12 : 15$). C sends its existing table vector to B, advertising that *C can reach Subnet X at cost 2*.
- **Loop Formation:** Router B receives C's vector. B incorrectly assumes C has an *alternate path* to Subnet X! B updates its table:

$$D_B(X) = c(B, C) + D_C(X) = 1 + 2 = 3 \quad (\text{Next Hop} = C) \quad (1)$$

- **Counting to Infinity:** At $t = 12 : 30$, B advertises $D_B(X) = 3$ to C. C updates its entry to $D_C(X) = 1 + 3 = 4$. This mutual incrementation loops continuously (3, 4, 5, 6...) until the metric reaches the protocol's defined infinity ($\infty = 16$ in RIP).

4.3 Distance Vector Iteration Table (Count-to-Infinity Trace)

5 Solutions to Routing Loops

5.1 1. Split Horizon Rule

Split Horizon Principle: A router **must never advertise a route back out the same interface** through which it originally learned that route.

Application: Because Router C learned its route to Subnet X through interface connected to B, Split Horizon prevents C from sending its entry for Subnet X back to Router B. This eliminates the loop at step t_1 .

Table 3: Step-by-Step Progression of Count-to-Infinity Loop for Destination Subnet X

Time Step	Event / Trigger	B's Distance to X	C's Distance to X	Loop Status
$t_0 = 12 : 00$	Link A-B Fails	∞ (Local)	2 (via B)	Normal State
$t_1 = 12 : 15$	C advertises to B	3 (via C)	2 (via B)	Loop Initiated!
$t_2 = 12 : 30$	B advertises to C	3 (via C)	4 (via B)	Incrementing...
$t_3 = 12 : 45$	C advertises to B	5 (via C)	4 (via B)	Incrementing...
t_N	Reaches Max Metric	16 (∞)	16 (∞)	Unreachable Declared

5.2 2. Poison Reverse (Split Horizon with Poison Reverse)

Instead of suppressing the route, **Poison Reverse** explicitly advertises the route back out the receiving interface with a distance metric set to **Infinity** ($\infty = 16$).

Table 4: Comparison of Split Horizon vs. Poison Reverse Update Generation

Mechanism	Update Sent by C to B regarding Subnet X	Action at Router B
Standard DV	Advertises $D_C(X) = 2$	Updates $D_B(X) = 3$ (Loop formed).
Split Horizon	Omits Subnet X entirely from vector	B retains $D_B(X) = \infty$ (No loop).
Poison Reverse	Explicitly advertises $D_C(X) = 16 (\infty)$	B immediately marks X as unreachable.

5.3 3. Timers and Route Poisoning

- **Route Poisoning:** When a link fails, the discovering router immediately sends a broadcast update declaring the affected network metric as 16 (∞).
- **Hold-Down Timers:** When a router receives an update setting a route to ∞ , it freezes that entry and refuses to accept any higher-cost updates for that network for a period (e.g., 180 seconds), allowing true failure status to propagate fully.

6 Solutions to Conceptual Exam Questions (Lecture Slides)

6.1 Question 1: Neighbor Tracking and Failure Detection

Exam Question: "Distance Vector algorithm does not keep track of their neighbors, why not? Explain how it will know that the neighbor is up or not?"

Detailed Answer:

1. **Why DV does not keep track of neighbor states:** Distance Vector protocols are designed to minimize memory and processing overhead on simple routers. They maintain only a routing table consisting of destination prefixes, cost metrics, and next-hop identifiers. DV does not maintain a separate topological neighbor state table (unlike OSPF, which maintains an explicit Neighbor Adjacency Table).
2. **How DV detects if a neighbor is UP or DOWN:** DV protocols rely on **periodic update timers and expiration timers**:

- Every router periodically transmits its full routing table to immediate neighbors (e.g., every 30 seconds in RIP).
- Each routing entry is bound to an **Invalidation / Expiration Timer** (typically 180 seconds).
- If a router fails to receive a periodic update from a neighbor for 180 consecutive seconds (6 missing update cycles), the router assumes the neighbor link is DOWN.
- The router then sets the metric for all routes learned through that neighbor to ∞ (16), marks them as unreachable, and propagates the failure.

6.2 Question 2: Absence of Hello Packets

Exam Question: "In the distance vector algorithm there is no exchange of hello packets, state why?"

Detailed Answer:

- **Implicit Keepalive Mechanism:** Link State protocols require lightweight **Hello Packets** to continuously verify link status because LS protocols send routing updates *only when topology changes occur*.
- In contrast, Distance Vector protocols send **full routing table updates periodically** (e.g., every 30s). These regular, periodic routing updates double as implicit keepalive signals. Sending separate Hello packets would be redundant and waste network bandwidth.

6.3 Question 3: Topology-Based First Iteration Vector Determination

Exam Question: "Refer to the topology given in Q2 (featuring R2 connected to PC A, PC B, R1 via s0/0/0, and R3 via s1/0/1). All routers run Distance Vector. Determine how R2 will know about its neighbors to send its information. In the first iteration, find out what kind of information R2 will send."

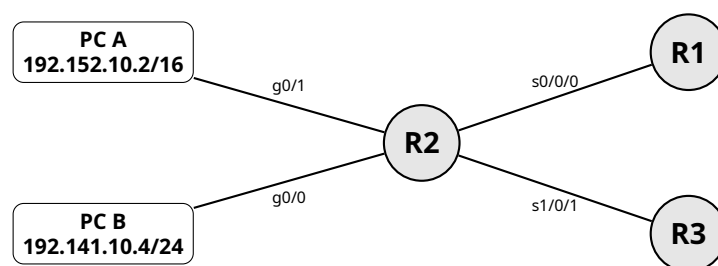


Figure 2: Exam Topology Excerpt Centered Around Router R2.

Detailed Solution:

1. **How R2 knows about its connected interfaces/neighbors:** Router R2 discovers directly connected networks and immediate links automatically through its **active physical and logical interfaces** (g0/1, g0/0, s0/0/0, s1/0/1) upon network boot up.
2. **Information sent by R2 in the first iteration ($t = 0s \rightarrow 30s$ update):** During the first update iteration, R2 transmits its initial **Directly Connected Network Vector** to its connected neighbors (R1 and R3). The transmitted payload contains:

- Subnet address of PC A network (192.152.0.0/16) – Hop Count = 0.
- Subnet address of PC B network (192.141.10.0/24) – Hop Count = 0.
- Network address of WAN Link s0/0/0 (connecting R2 to R1) – Hop Count = 0.
- Network address of WAN Link s1/0/1 (connecting R2 to R3) – Hop Count = 0.